

LAPORAN WHITE BOX TESTING — CODE WALKTHROUGH

Aplikasi BABEPUS (Barang Bekas Puspita)

1. Pendahuluan & Konsep Code Walkthrough

Code Walkthrough adalah metode pengujian White Box yang dilakukan dengan cara menelusuri kode program secara manual, baris per baris, mengikuti alur eksekusi nyata dari sebuah skenario penggunaan. Berbeda dengan Data Flow Testing dan Control Flow Testing yang bersifat analitis per fungsi, Code Walkthrough mensimulasikan perjalanan request pengguna mulai dari client (React.js), melewati lapisan middleware, controller, service, hingga database — dan kembali sebagai response.

1.1 Tujuan Code Walkthrough

- Memverifikasi bahwa setiap lapisan kode (route → middleware → controller → service → DB) berjalan sesuai urutan yang benar.
- Mendeteksi potensi bug logika yang tidak terlihat dari pengujian unit per fungsi.
- Memastikan penanganan error (ApiError, try-catch, rollback) berjalan di lapisan yang tepat.
- Mendokumentasikan alur kode yang bisa digunakan sebagai referensi tim pengembang.

1.2 Arsitektur Lapisan Aplikasi BABEPUS

Layer	File / Komponen	Tanggung Jawab
Client (Frontend)	React.js + Axios/Fetch	Mengirim HTTP request dengan JWT Bearer Token di header Authorization
Router	src/routes/*.js	Mendaftarkan endpoint URL dan menentukan urutan middleware yang berjalan
Middleware Auth	authMiddleware.js	Verifikasi JWT, SELECT user dari DB, isi req.user, atau throw 401/403
Middleware Validate	validateRequest.js + express-validator	Validasi format input — jika gagal throw ApiError(422) dengan detail field
Middleware Upload	uploadMiddleware.js (multer)	Proses file upload, validasi MIME type dan ukuran, simpan ke /uploads/
Controller	src/controllers/*.js	Terima req, delegasikan ke service, wrap dengan asyncHandler untuk error propagation
Service	src/services/*.js	Logika bisnis, query DB dengan parameterized query, throw ApiError jika gagal
Utils	ApiError, asyncHandler, jwt, password, pagination, slug, file	Helper lintas lapisan
Database	MySQL via mysql2/promise pool	Eksekusi query, transaksi (beginTransaction/commit/rollback), connection pooling

Layer	File / Komponen	Tanggung Jawab
Error Handler	errorHandler.js	Tangkap semua error, normalisasi MulterError dan ER_DUP_ENTRY, kirim response JSON

2. Daftar Skenario Code Walkthrough

Enam skenario dipilih berdasarkan kompleksitas alur dan kritikalitas fitur terhadap keamanan dan integritas data aplikasi BABEPUS:

No	Skenario	Modul yang Terlibat	Kompleksitas
1	Register & Login Pengguna	authRoutes → authMiddleware → authController → authService → DB	Tinggi
2	Membuat dan Mempublikasikan Produk	productRoutes → authMiddleware → uploadMiddleware → productController → productService → DB	Tinggi
3	Mengirim Penawaran (Offer)	offerRoutes → authMiddleware → validateRequest → offerController → offerService → DB Transaction	Sangat Tinggi
4	Menerima Penawaran & Membuat Transaksi	offerRoutes → offerService → transactionService → notificationService → DB Transaction	Sangat Tinggi
5	Konfirmasi Escrow & Auto-Complete Transaksi	transactionRoutes → authMiddleware → transactionController → transactionService → DB Transaction	Sangat Tinggi
6	Pencarian Produk & Fitur Wishlist	productRoutes → optionalAuthMiddleware → productController → productService + wishlistService → DB	Sedang

3. Walkthrough Skenario 1: Register & Login Pengguna

Skenario: Mahasiswa baru mendaftarkan akun, lalu login untuk mendapatkan token JWT.

3.1 Walkthrough Register (POST /api/auth/register)

Langka h	Layer	File	Kode / Aksi	Output / State
1	Router	authRoutes.js	router.post('/register', registerValidator, validateRequest, authController.register)	Request masuk, chain middleware dimulai
2	Middlewar e	express-validator	registerValidator: cek fullName, email format, password min 8 char, campus, studentId	Jika invalid → next(ApiError(422)) → errorHandler
3	Middlewar e	validateRequest.js	validationResult(req) → jika errors.isEmpty() = false →	Jika valid → lanjut ke controller

Langka h	Layer	File	Kode / Aksi	Output / State
			next(ApiError(422, errors))	
4	Controller	authController.js	asyncHandler: const data = await authService.register(req.body)	Delegasi ke service, error otomatis ke next()
5	Service	authService.js	SELECT users WHERE email = ? → existingUsers.length > 0 → throw ApiError(409)	Jika email duplikat → 409 Conflict
6	Service	authService.js	hashPassword(payload.password) → bcrypt.hash(password, 12)	passwordHash tersimpan, plain password tidak pernah ke DB
7	Service	authService.js	generateEmailVerificationToken() → crypto.randomBytes(32).toString('hex')	Token verifikasi 64 karakter hex
8	Service	authService.js	pool.getConnection() → beginTransaction()	Transaksi DB dimulai — atomik
9	DB	MySQL	INSERT INTO users (full_name, email, password_hash, ..., 'pending', token, expiresAt)	User baru tersimpan, status = pending
10	DB	MySQL	INSERT INTO verifications (user_id, document_type, student_id, campus_email, 'pending')	Record verifikasi mahasiswa dibuat
11	Service	authService.js	connection.commit() → findUserById(result.insertId) → signToken({ sub: user.id, role })	Transaksi berhasil, JWT dibuat
12	Service	authService.js	return { token, user, emailVerification: { token, expiresAt } }	Data lengkap dikembalikan
13	Controller	authController.js	res.status(201).json({ success: true, message, data })	Response 201 dikirim ke client
14	Error (alt)	authService.js	catch(error) → connection.rollback() → throw error	Jika gagal → rollback → errorHandler

```
// Alur register – ringkasan kode aktual
// authRoutes.js
router.post('/register', registerValidator, validateRequest,
authController.register);

// authController.js
const register = asyncHandler(async (req, res) => {
  const data = await authService.register(req.body); // delegasi ke service
  res.status(201).json({ success: true, data });
});

// authService.js – register()
const [existingUsers] = await pool.query('SELECT id FROM users WHERE email = ?',
[email]);
if (existingUsers.length) throw new ApiError(409, 'Email sudah terdaftar.');
```

```
const passwordHash = await hashPassword(payload.password); // bcrypt salt=12
const connection = await pool.getConnection();
```

```

try {
  await connection.beginTransaction();
  await connection.query('INSERT INTO users ...', [...]);
  await connection.query('INSERT INTO verifications ...', [...]);
  await connection.commit();
  return { token: signToken({...}), user, emailVerification };
} catch (e) { await connection.rollback(); throw e; }
finally { connection.release(); }

```

3.2 Walkthrough Login (POST /api/auth/login)

Langkah	Layer	File	Kode / Aksi	Output / State
1	Router	authRoutes.js	router.post('/login', loginValidator, validateRequest, authController.login)	Request masuk, validasi email & password
2	Middleware	validateRequest.js	Cek errors.isEmpty() → lanjut jika valid	Jika invalid → 422 dengan detail field error
3	Controller	authController.js	const data = await authService.login(req.body)	Delegasi ke service
4	Service	authService.js	SELECT ... FROM users WHERE email = ? LIMIT 1	rows = hasil query
5	Service	authService.js	if (!rows.length) throw ApiError(401, '...')	Email tidak ditemukan → 401
6	Service	authService.js	if (rows[0].isSuspended) throw ApiError(403, '...')	Akun suspended → 403
7	Service	authService.js	const isPasswordValid = await comparePassword(password, rows[0].passwordHash)	bcrypt.compare() — bisa 100-200ms
8	Service	authService.js	if (!isPasswordValid) throw ApiError(401, '...')	Password salah → 401
9	Service	authService.js	const user = serializeUser(rows[0]) → const token = signToken({ sub: user.id, role })	JWT ditandatangani dengan JWT_SECRET, expire 7d
10	Controller	authController.js	res.json({ success: true, message: 'Login berhasil', data: { token, user } })	Response 200 + token dikirim ke client

3.3 Walkthrough GET /api/auth/me (Akses Data User Sendiri)

Langka h	Layer	File	Kode / Aksi	Output / State
1	Router	authRoutes.js	router.get('/me', authMiddleware, authController.me)	Wajib login

Langka h	Layer	File	Kode / Aksi	Output / State
2	Middlewar e	authMiddleware. js	getBearerToken(req.headers.authorization) → split 'Bearer <token>'	Ambil token dari header
3	Middlewar e	authMiddleware. js	if (!token) throw ApiError(401)	Tidak ada token → 401
4	Middlewar e	authMiddleware. js	verifyToken(token) → jwt.verify(token, JWT_SECRET)	Jika invalid/expired → JsonWebTokenError → ApiError(401)
5	Middlewar e	authMiddleware. js	SELECT ... FROM users WHERE id = payload.sub LIMIT 1	Ambil data user terbaru dari DB
6	Middlewar e	authMiddleware. js	if (!rows.length) throw ApiError(401) if (isSuspended) throw ApiError(403)	Cek keabsahan dan status user
7	Middlewar e	authMiddleware. js	req.user = rows[0]; return next()	req.user terisi, lanjut ke controller
8	Controller	authController.js	const user = await authService.findUserById(req.user.id)	Ambil data lengkap user
9	Controller	authController.js	res.json({ success: true, data: { user } })	Response 200 + profil user

4. Walkthrough Skenario 2: Membuat Produk Baru

Skenario: Pengguna yang sudah login mengunggah foto produk dan mengisi detail produk baru untuk dipublikasikan di marketplace.

4.1 Walkthrough POST /api/products

Langka h	Layer	File	Kode / Aksi	Output / State
1	Router	productRoutes.js	router.post('/', authMiddleware, productImageUpload, productPayloadValidator, validateRequest, productController.createProduct)	Chain: auth → upload → validasi → controller
2	Middlewar e	authMiddleware.js	Verifikasi JWT, SELECT user, isi req.user	req.user terisi atau 401/403
3	Middlewar e	uploadMiddleware.js	multer.diskStorage: validasi MIME (jpeg/png/webp), max 5MB, sanitasi nama file	File disimpan ke /uploads/ dengan nama aman
4	Middlewar e	uploadMiddleware.js	fileFilter: cek allowedMimeTypes.includes(file.mimetype)	Jika tidak valid → ApiError(415)

Langka h	Layer	File	Kode / Aksi	Output / State
				)
5	Middlewar e	uploadMiddleware.js	filename: baseName-timestamp- random.ext	Nama file acak mencegah collision dan path traversal
6	Middlewar e	validateRequest.js	Cek errors dari productPayloadValidator: title, categoryId, price, conditionLabel wajib	Jika invalid → 422 dengan detail field
7	Controller	productController.js	if (!req.file) throw new ApiError(422, 'Gambar produk wajib diunggah.')	Guard: pastikan file ada setelah multer
8	Controller	productController.js	const imageUrl = getUploadUrl(req.file.filename) → 'uploads/namafile.jpg'	URL relatif untuk disimpan di DB
9	Controller	productController.js	await productService.createProduct(req.user.id , req.body, imageUrl)	Delegasi ke service
10	Service	productService.js	await ensureCategoryExists(payload.categoryId) → SELECT categories WHERE id	Jika kategori tidak ada → ApiError(422)
11	Service	productService.js	const slug = await generateUniqueSlug(payload.title)	while(true): cek slug, tambah suffix jika perlu
12	DB	MySQL	INSERT INTO products (seller_id, category_id, title, slug, ..., status='active')	Produk tersimpan dengan status aktif
13	Service	productService.js	return getProductById(result.insertId, sellerId)	Ambil data produk lengkap dengan JOIN
14	Controller	productController.js	res.status(201).json({ success: true, message, data: { product } })	Response 201 + data produk dikirim

```
// productRoutes.js - chain middleware createProduct
router.post('/',
  authMiddleware,      // Step 2: verifikasi JWT
  productImageUpload,  // Step 3-5: multer upload
```

```

    productPayloadValidator, // Step 6: validasi input
    validateRequest,        // Step 6: cek error validasi
    productController.createProduct // Step 7-14
  );

// uploadMiddleware.js - filename sanitizer
filename: (_req, file, callback) => {
  const extension = path.extname(file.originalname).toLowerCase();
  const baseName = path.basename(file.originalname, extension).replace(/[^a-zA-Z0-9]/g, '_');
  const safeName = `${baseName}-${Date.now()}-${Math.round(Math.random() * 1e9)}${extension}`;
  callback(null, safeName);
}

// productService.js - generateUniqueSlug()
while (true) {
  const [rows] = await pool.query('SELECT id FROM products WHERE slug = ? LIMIT 1', [candidate]);
  if (!rows.length) return candidate; // slug tersedia
  suffix += 1;
  candidate = `${baseSlug}-${suffix}`; // coba dengan suffix
}

```

5. Walkthrough Skenario 3: Mengirim Penawaran (Offer)

Skenario: Pembeli mengirim tawaran harga untuk sebuah produk aktif milik penjual lain. Ini adalah alur paling kompleks karena melibatkan DB transaction, multiple validation, dan race condition prevention.

5.1 Walkthrough POST /api/offers

Langkah	Layer	File	Kode / Aksi	Output / State
1	Router	offerRoutes.js	router.use(authMiddleware) → router.post('/', createOfferValidator, validateRequest, offerController.createOffer)	Semua route wajib login
2	Middleware	authMiddleware.js	Verifikasi JWT → req.user terisi	req.user = { id, role, ... }
3	Middleware	validateRequest.js	createOfferValidator: productId integer, offerPrice number min 1, note optional	Jika invalid → 422
4	Controller	offerController.js	const offer = await offerService.createOffer(req.user.id, req.body)	Delegasi ke service
5	Service	offerService.js	const connection = await pool.getConnection() → beginTransaction()	Transaksi DB dimulai — FOR UPDATE mencegah race condition
6	DB	MySQL	SELECT products ... WHERE id = ? FOR UPDATE — kunci baris produk	products[0] = data produk terkunci

Langkah	Layer	File	Kode / Aksi	Output / State
7	Service	offerService.js	if (!products.length) throw ApiError(404)	Produk tidak ada → 404 → rollback
8	Service	offerService.js	if (product.status !== 'active') throw ApiError(422)	Produk tidak aktif → 422 → rollback
9	Service	offerService.js	if (product.sellerId === buyerId) throw ApiError(422)	Tidak bisa tawar sendiri → 422 → rollback
10	Service	offerService.js	if (product.sellerSuspended) throw ApiError(422)	Seller suspended → 422 → rollback
11	DB	MySQL	SELECT offers WHERE product_id=? AND buyer_id=? AND status='pending' FOR UPDATE	Cek pending offer
12	Service	offerService.js	if (existingOffers.length) throw ApiError(409)	Sudah ada pending offer → 409 → rollback
13	Service	offerService.js	if (offerPrice >= product.price) throw ApiError(422)	Harga >= harga jual → 422 → rollback
14	Service	offerService.js	if (offerPrice < 10000) throw ApiError(422)	Harga < Rp10.000 → 422 → rollback
15	DB	MySQL	INSERT INTO offers (product_id, buyer_id, seller_id, offer_price, note, 'pending')	Offer tersimpan
16	Service	offerService.js	await connection.commit()	Transaksi selesai, data aman
17	Service	offerService.js	await getOfferById(result.insertId) → SELECT offer JOIN products, buyer, seller	Data offer lengkap diambil
18	Service	offerService.js	await createNotification({ userId: product.sellerId, type: 'offer', ... })	Notifikasi real- time dikirim ke seller via SSE
19	Controller	offerController.js	res.status(201).json({ success: true, data: { offer } })	Response 201 + data offer
20	Error (alt)	offerService.js	catch(error) → connection.rollback() → throw error → errorHandler	Semua perubahan dibatalkan jika ada error

```
// offerService.js - createOffer() - transaction + 8 validasi + notification
const connection = await pool.getConnection();
try {
```



```

    await connection.beginTransaction();
    const [products] = await connection.query( // FOR UPDATE: lock row
      'SELECT p.*, u.is_suspended AS sellerSuspended FROM products p
      INNER JOIN users u ON u.id = p.seller_id
      WHERE p.id = ? AND p.deleted_at IS NULL LIMIT 1 FOR UPDATE',
      [payload.productId]
    );
    if (!products.length) throw new ApiError(404, '...'); // Guard 1
    if (status !== 'active') throw new ApiError(422, '...'); // Guard 2
    if (sellerId === buyerId) throw new ApiError(422, '...'); // Guard 3
    if (sellerSuspended) throw new ApiError(422, '...'); // Guard 4
    // cek existing pending offer...
    if (existingOffers.length) throw new ApiError(409, '...'); // Guard 5
    if (offerPrice >= product.price) throw new ApiError(422, '...'); // Guard 6
    if (offerPrice < 10000) throw new ApiError(422, '...'); // Guard 7
    await connection.query('INSERT INTO offers ...', [...]);
    await connection.commit();
    // ...getOfferById + createNotification
  } catch (e) { await connection.rollback(); throw e; }
  finally { connection.release(); }

```

6. Walkthrough Skenario 4: Menerima Penawaran & Membuat Transaksi

Skenario: Penjual menerima penawaran dari pembeli. Sistem secara otomatis membuat transaksi, menandai produk terjual, menolak offer lain yang pending, dan mengirim notifikasi.

6.1 Walkthrough PATCH /api/offers/:id/accept

Langkah	Layer	File	Kode / Aksi	Output / State
1	Router	offerRoutes.js	router.patch('/:id/accept', idParam('id'), validateRequest, offerController.acceptOffer)	Validasi id integer, wajib login
2	Middleware	authMiddleware.js	Verifikasi JWT → req.user = seller	req.user berisi data penjual
3	Controller	offerController.js	const data = await offerService.acceptOffer(req.params.id, req.user.id)	Delegasi ke service
4	Service	offerService.js	pool.getConnection() → beginTransaction()	Transaksi atomik dimulai
5	DB	MySQL	SELECT offers JOIN products WHERE o.id=? AND o.seller_id=? FOR UPDATE	Kunci baris offer dan produk
6	Service	offerService.js	if (!offers.length) throw ApiError(404)	Offer tidak ada atau bukan milik seller → 404
7	Service	offerService.js	if (offer.offerStatus !== 'pending') throw ApiError(422)	Offer sudah diproses → 422

Langkah	Layer	File	Kode / Aksi	Output / State
8	Service	offerService.js	if (offer.deletedAt productStatus !== 'active') throw ApiError(422)	Produk tidak tersedia → 422
9	DB	MySQL	UPDATE offers SET status='accepted' WHERE id=?	Offer diterima
10	DB	MySQL	UPDATE offers SET status='auto_rejected' WHERE product_id=? AND id<>? AND status='pending'	Semua offer pending lain ditolak otomatis
11	DB	MySQL	UPDATE products SET status='sold', sold_at=NOW() WHERE id=?	Produk ditandai terjual
12	DB	MySQL	INSERT INTO transactions (offer_id, product_id, buyer_id, seller_id, final_price, 'pending_meetup')	Transaksi escrow otomatis dibuat
13	Service	offerService.js	await connection.commit()	Semua perubahan tersimpan atomik
14	Service	notificationService.js	createNotification({ userId: buyer.id, type: 'offer_accepted', ... })	Notifikasi ke pembeli via SSE
15	Controller	offerController.js	res.json({ success: true, data: { offer, transactionId } })	Response 200 + offer + transactionId

```
// offerService.js - acceptOffer() - 4 operasi DB dalam 1 transaksi
await connection.query("UPDATE offers SET status='accepted' WHERE id=?", [offerId]);
await connection.query( // Auto-reject semua offer lain
  "UPDATE offers SET status='auto_rejected'
  WHERE product_id=? AND id<>? AND status='pending'",
  [offer.productId, offerId]
);
await connection.query( // Tandai produk terjual
  "UPDATE products SET status='sold', sold_at=NOW() WHERE id=?",
  [offer.productId]
);
await connection.query( // Buat transaksi escrow
  "INSERT INTO transactions (offer_id, product_id, buyer_id, seller_id, final_price,
status)
  VALUES (?, ?, ?, ?, ?, 'pending_meetup')",
  [offerId, offer.productId, offer.buyerId, offer.sellerId, offer.offerPrice]
);
await connection.commit(); // Semua atau tidak sama sekali
```

7. Walkthrough Skenario 5: Konfirmasi Escrow & Auto-Complete Transaksi

Skenario: Setelah COD, pembeli mengonfirmasi barang diterima. Jika penjual juga sudah konfirmasi (atau sebaliknya), transaksi otomatis selesai dan escrow dirilis. Ini adalah logika bisnis paling kritis di aplikasi.

7.1 Walkthrough PATCH /api/transactions/:id/escrow/buyer-confirm

Langka h	Layer	File	Kode / Aksi	Output / State
1	Router	transactionRoutes.js	router.use(authMiddleware) → router.patch('/:id/escrow/buyer-confirm', completeTransactionValidator, validateRequest, transactionController.confirmBuyer)	Wajib login, validasi id
2	Controll er	transactionController.js	const transaction = await transactionService.confirmEscrow(req.params.id, req.user.id, 'buyer')	Delegasi dengan role='buyer'
3	Service	transactionService.js	pool.getConnection() → beginTransaction()	Transaksi DB dimulai
4	DB	MySQL	SELECT ... FROM transactions WHERE id=? FOR UPDATE	Kunci baris transaksi
5	Service	transactionService.js	if (!rows.length) throw ApiError(404)	Transaksi tidak ada → 404
6	Service	transactionService.js	if (![buyerId, sellerId].includes(userId)) throw ApiError(403)	User bukan bagian transaksi → 403
7	Service	transactionService.js	if (status !== 'pending_meetup' escrowStatus !== 'holding') throw ApiError(422)	Status tidak valid → 422
8	Service	transactionService.js	if (role === 'buyer' && buyerId !== userId) throw ApiError(403)	Hanya pembeli yang bisa konfirmasi buyer
9	Service	transactionService.js	const field = 'buyer_confirmed_at', eventType = 'buyer_confirmed'	Tentukan field dan event berdasarkan role
10	DB	MySQL	UPDATE transactions SET buyer_confirmed_at = COALESCE(buyer_confirmed_at, NOW()) WHERE id=?	COALESCE: hanya update jika belum ada (idempoten)
11	DB	MySQL	INSERT INTO escrow_events (transaction_id, actor_id, 'buyer_confirmed', 'Pembeli mengonfirmasi...')	Log audit escrow

Langka h	Layer	File	Kode / Aksi	Output / State
12	DB	MySQL	SELECT buyer_confirmed_at, seller_confirmed_at FROM transactions WHERE id=?	Cek apakah kedua pihak sudah konfirmasi
13a	Service	transactionService.js	if (buyerConfirmedAt && sellerConfirmedAt) → UPDATE status='completed', escrow='released'	JALUR AUTO- COMPLET E: kedua sudah konfirmasi
13b	Service	transactionService.js	INSERT INTO escrow_events (..., 'escrow_released', 'Dana escrow otomatis dirilis...')	Log rilis dana escrow
14	Service	transactionService.js	await connection.commit()	Semua perubahan tersimpan
15	Service	notificationService.js	createNotification({ userId: sellerId, type: 'escrow', title: 'Pembeli mengonfirmasi barang' })	Notifikasi ke seller
16	Controll er	transactionController .js	res.json({ success: true, data: { transaction } })	Response 200 + state transaksi terbaru

```
// transactionService.js - confirmEscrow() - logika auto-complete
const field = role === 'buyer' ? 'buyer_confirmed_at' : 'seller_confirmed_at';
const eventType = role === 'buyer' ? 'buyer_confirmed' : 'seller_confirmed';

// COALESCE: idempoten - tidak overwrite jika sudah ada
await connection.query(
  `UPDATE transactions SET ${field} = COALESCE(${field}, NOW()) WHERE id = ?`,
  [transactionId]
);

// Cek apakah kedua pihak sudah konfirmasi
const [updatedRows] = await connection.query(
  'SELECT buyer_confirmed_at, seller_confirmed_at FROM transactions WHERE id = ?',
  [transactionId]
);

// AUTO-COMplete: jika keduanya sudah konfirmasi
if (updatedRows[0].buyerConfirmedAt && updatedRows[0].sellerConfirmedAt) {
  await connection.query(
    `UPDATE transactions SET status='completed', escrow_status='released',
    completed_at=NOW(), payout_released_at=NOW() WHERE id=?`,
    [transactionId]
  );
  await connection.query( // Audit log
    'INSERT INTO escrow_events (... , escrow_released, Dana escrow otomatis
    dirilis...)',
    [transactionId, userId]
  );
}
await connection.commit();
```

8. Walkthrough Skenario 6: Pencarian Produk & Fitur Wishlist

Skenario: Pengguna mencari produk dengan filter, melihat detail, lalu menambahkannya ke wishlist. Menggunakan optionalAuthMiddleware — bisa diakses tanpa login, tapi dengan fitur tambahan jika login.

8.1 Walkthrough GET

/api/products?search=laptop&categoryId=1&maxPrice=500000

Langkah	Layer	File	Kode / Aksi	Output / State
1	Router	productRoutes.js	router.get('/', optionalAuthMiddleware, productListValidator, paginationQuery, validateRequest, productController.listProducts)	Optional auth — bisa tanpa token
2	Middleware	authMiddleware.js (optional)	getBearerToken → jika tidak ada token → next() langsung (tanpa error)	req.user = undefined jika tidak login
3	Middleware	authMiddleware.js (optional)	Jika ada token → verifyToken → SELECT user → jika valid dan tidak suspended → req.user = rows[0]	req.user terisi jika token valid
4	Middleware	validateRequest.js	Validasi query params: search string, categoryId integer, maxPrice number	Jika invalid → 422
5	Controller	productController.js	const currentUserId = req.user?.id null	null jika tidak login
6	Controller	productController.js	await productService.getProducts(req.query, currentUserId)	Delegasi ke service
7	Service	productService.js	getPagination(query) → page=1, limit=12 (max 50), offset=0	Paginasi dihitung
8	Service	productService.js	conditions = ['deleted_at IS NULL', 'status=active', 'is_suspended=0']	Kondisi dasar always active
9	Service	productService.js	if (query.search) → conditions.push(LIKE) → keyword = '%laptop%'	Filter teks ditambahkan
10	Service	productService.js	if (query.categoryId) → conditions.push('category_id=?')	Filter kategori ditambahkan
11	Service	productService.js	if (query.maxPrice) → conditions.push('price <= ?')	Filter harga ditambahkan
12	Service	productService.js	if (currentUserId) → conditions.push('seller_id <> ?') + isWishlisted subquery	Sembunyikan produk sendiri + cek wishlist jika login
13	DB	MySQL	SELECT COUNT(*) AS total → untuk	total = jumlah

Langkah	Layer	File	Kode / Aksi	Output / State
			meta paginasi	produk yang cocok
14	DB	MySQL	SELECT p.*, c.*, u.*, (isWishlisted subquery) ORDER BY created_at DESC LIMIT 12 OFFSET 0	Data produk halaman pertama
15	Service	productService.js	rows.map(formatProduct) → konversi tipe data (Number, Boolean)	Array produk terformat
16	Service	productService.js	buildPaginationMeta({ page:1, limit:12, total }) → { totalPages: Math.ceil(total/limit) }	Meta paginasi
17	Controller	productController.js	res.json({ success: true, data: { products }, meta })	Response 200 + produk + meta

8.2 Walkthrough POST /api/wishlist/:productId (Tambah ke Wishlist)

Langkah	Layer	File	Kode / Aksi	Output / State
1	Router	wishlistRoutes.js	router.use(authMiddleware) → router.post('/:productId', wishlistProductValidator, validateRequest, wishlistController.addToWishlist)	Wajib login
2	Middleware	authMiddleware.js	Verifikasi JWT → req.user terisi	Jika tidak ada token → 401
3	Controller	wishlistController.js	await wishlistService.addToWishlist(req.user.id, req.params.productId)	Delegasi ke service
4	Service	wishlistService.js	SELECT products WHERE id=? AND deleted_at IS NULL LIMIT 1	Cek produk ada
5	Service	wishlistService.js	if (!products.length) throw ApiError(404)	Produk tidak ada → 404
6	Service	wishlistService.js	if (products[0].sellerId === userId) throw ApiError(422)	Tidak bisa wishlist produk sendiri → 422
7	Service	wishlistService.js	if (products[0].status !== 'active') throw ApiError(422)	Produk tidak aktif → 422
8	DB	MySQL	INSERT IGNORE INTO wishlists (user_id, product_id) VALUES (?, ?)	IGNORE: tidak error jika sudah ada

Langkah	Layer	File	Kode / Aksi	Output / State
				(idempoten)
9	Controller	wishlistController.js	res.status(201).json({ data: { wishlist: { productId, isWishlisted: true } } })	Response 201 + status wishlist

```
// optionalAuthMiddleware - tidak wajib token
const optionalAuthMiddleware = async (req, _res, next) => {
  const token = getBearerToken(req.headers.authorization);
  if (!token) return next(); // Tidak ada token - lanjut tanpa user
  try {
    const payload = verifyToken(token);
    const [rows] = await pool.query('SELECT id, role, is_suspended FROM users WHERE id=?', [payload.sub]);
    if (rows.length && !rows[0].isSuspended) req.user = rows[0]; // Isi req.user jika valid
  } catch (_error) { /* token invalid - abaikan, lanjut tanpa user */ }
  return next();
};

// productService.js - query dinamis berdasarkan currentUserId
const queryParams = currentUserId
  ? [currentUserId, ...params, limit, offset] // currentUserId untuk isWishlisted subquery
  : [...params, limit, offset]; // tanpa userId

// wishlistService.js - INSERT IGNORE (idempoten)
await pool.query(
  'INSERT IGNORE INTO wishlists (user_id, product_id) VALUES (?, ?)',
  [userId, productId]
);
return { productId, isWishlisted: true }; // Selalu return true meski duplikat
```

9. Walkthrough Error Handler & Middleware Global

Semua error yang terjadi di semua lapisan diteruskan ke middleware errorHandler melalui next(error). Berikut alur penanganan error secara global:

9.1 asyncHandler — Error Propagation

```
// utils/asyncHandler.js - wrapper agar error async terkirim ke next()
const asyncHandler = (handler) => (req, res, next) => {
  Promise.resolve(handler(req, res, next)).catch(next);
  // Jika handler throw, .catch(next) → next(error) → errorHandler
};

// Tanpa asyncHandler, error async tidak akan terkirim ke Express error middleware
```

9.2 errorHandler — Normalisasi & Response Error

Jenis Error	Deteksi	Normalisasi	Status Code	Response
ApiError (custom)	instanceof ApiError	Tidak diubah —	Sesuai	{ success: false,

Jenis Error	Deteksi	Normalisasi	Status Code	Response
		gunakan statusCode & message	ApiError	message }
MulterError LIMIT_FILE_SIZE	instanceof multer.MulterError && code === LIMIT_FILE_SIZE	new ApiError(413, 'Ukuran gambar terlalu besar.')	413	{ success: false, message }
MulterError lainnya	instanceof multer.MulterError	new ApiError(400, 'Upload gambar gagal.')	400	{ success: false, message }
MySQL ER_DUP_ENTRY	error.code === 'ER_DUP_ENTRY'	new ApiError(409, 'Data yang sama sudah terdaftar.')	409	{ success: false, message }
JsonWebTokenError	Ditangkap di authMiddleware catch	new ApiError(401, 'Token tidak valid.')	401	{ success: false, message }
Error 500 (production)	statusCode >= 500 && NODE_ENV = production	stack tidak disertakan di response	500	{ success: false, message }
Error 500 (development)	statusCode >= 500 && NODE_ENV != production	stack disertakan di response	500	{ success: false, message, stack }
ApiError dengan details	normalizedError.details	errors: [...] disertakan	422	{ success: false, message, errors: [{}field, message]] }

```
// errorHandler.js - normalisasi semua error ke JSON response
const errorHandler = (error, _req, res, _next) => {
  let normalizedError = error;

  if (error instanceof multer.MulterError) { // Normalisasi multer error
    normalizedError = new ApiError(
      error.code === 'LIMIT_FILE_SIZE' ? 413 : 400,
      error.code === 'LIMIT_FILE_SIZE' ? 'Ukuran gambar terlalu besar.' : 'Upload gambar gagal.'
    );
  }
  if (error?.code === 'ER_DUP_ENTRY') { // Normalisasi MySQL duplicate error
    normalizedError = new ApiError(409, 'Data yang sama sudah terdaftar.');
```


10. Temuan & Rekomendasi dari Code Walkthrough

No	Temuan	Lokasi	Kategori	Rekomendasi
1	Semua route sensitif sudah dilindungi authMiddleware atau router.use(authMiddleware)	Semua *Routes.js	Positif	Pertahankan — tidak ada endpoint yang lupa dilindungi
2	asyncHandler membungkus semua controller — tidak ada unhandled promise rejection	Semua *Controller.js	Positif	Pertahankan pattern ini untuk semua handler baru
3	Operasi kritis (register, offer, acceptOffer, confirmEscrow, review) menggunakan DB transaction dengan rollback	authService, offerService, transactionService, reviewService	Positif	Pertahankan — integritas data terjaga
4	FOR UPDATE digunakan saat operasi kritis — mencegah race condition pada concurrent request	offerService, transactionService	Positif	Pertahankan — penting untuk sistem transaksi real
5	JWT_SECRET default 'development_secret_change_me' diblokir di production via env.js	config/env.js	Positif	Sudah ada guard — pastikan di-set di environment production
6	createOffer memanggil connection.commit() dua kali (baris 1 setelah INSERT, baris 2 setelah createNotification)	offerService.js	Bug Minor	Hapus salah satu commit() — commit ganda tidak menyebabkan crash tapi merupakan dead code
7	uploadMiddleware menyimpan file ke disk sebelum validasi input dijalankan — file bisa tersisa jika validasi gagal	productRoutes.js, uploadMiddleware.js	Perhatian	Tambahkan cleanup file jika validateRequest gagal, atau gunakan memoryStorage + simpan setelah validasi
8	Pencarian menggunakan LIKE '%keyword%' tanpa full-text index — bisa lambat untuk data besar	productService.js getProducts()	Performa	Pertimbangkan FULLTEXT INDEX pada kolom title dan description untuk skala produksi
9	optionalAuthMiddleware mengabaikan semua error token (catch kosong) — token expired pun dianggap tidak login	authMiddleware.js	Perilaku	Sudah tepat untuk endpoint publik — hanya perlu dokumentasi agar developer tidak keliru ekspektasi
10	COALESCE pada confirmEscrow memastikan idempoten — konfirmasi ganda tidak overwrite timestamp pertama	transactionService.js	Positif	Pertahankan — penting untuk konsistensi data escrow

11. Kesimpulan Code Walkthrough

11.1 Ringkasan Skenario yang Diuji

No	Skenario	Lapisan yang Dilalui	Hasil
1	Register & Login	Route → Validator → asyncHandler → authService → bcrypt → JWT → DB Transaction	Lulus
2	Membuat Produk	Route → authMiddleware → multer upload → Validator → productService → slug generator → DB	Lulus
3	Mengirim Penawaran	Route → authMiddleware → Validator → offerService → DB Transaction (FOR UPDATE) → 8 validasi → Notifikasi	Lulus
4	Menerima Penawaran	Route → authMiddleware → offerService → 4 DB operations atomik → auto-reject offers lain → Notifikasi	Lulus
5	Konfirmasi Escrow	Route → authMiddleware → transactionService → DB Transaction → COALESCE → auto-complete logic → Notifikasi	Lulus
6	Pencarian & Wishlist	Route → optionalAuthMiddleware → dynamic WHERE builder → paginasi → INSERT IGNORE	Lulus

11.2 Kesimpulan Akhir

1. Code Walkthrough berhasil dilakukan terhadap 6 skenario end-to-end yang mencakup seluruh 13 modul utama aplikasi BABEPUS.
2. Arsitektur layered (Route → Middleware → Controller → Service → DB) diterapkan secara konsisten di seluruh modul tanpa pengecualian.
3. Mekanisme keamanan (JWT verification, bcrypt, parameterized query, DB transaction + rollback, FOR UPDATE) sudah terimplementasi dengan benar di lapisan yang tepat.
4. Ditemukan 1 bug minor (double commit di offerService) dan 1 potensi perbaikan (file cleanup saat validasi gagal) yang dapat diperbaiki tanpa breaking change.
5. Aplikasi BABEPUS dinyatakan LULUS Code Walkthrough dengan kualitas kode yang baik, struktur yang konsisten, dan penanganan error yang menyeluruh di semua lapisan.